

Scientific Python

Batteries Included

Ianna Osborne, CoDaS-HEP 2026

Evolution of Programming Languages

Programming languages evolved in distinct waves — from punched-card machine instructions to AI-assisted coding.

Punched Cards & Machine Code
(1840s–1950s)



Low-Level
Programming

Assembly Languages
(1947–1950s)



Symbolic Instructions

Early Compiled
High-Level Languages
(1952–1965)



High-Level Compilers

Structured
Programming
& Systems
Languages
(1969–1979)



C & Object-Oriented

Interpreted
Languages &
Virtual Machines
(1980s–2000s)



Web & Scripting

Modern Multi-Paradigm &
Memory-Safe Languages
(2010s–2020s)



Safe & Concurrent

AI-Assisted &
AI-Native
Programming
(2020s–2026)



AI-Driven
Development

Programming languages evolved through distinct eras, each introducing new abstractions that reshaped how humans communicate intent to machines.

- Analytical Engine algorithms (1840s): Ada Lovelace's Bernoulli number algorithm is considered the first published computer program.
- Jacquard cards (1804 → influence through 19th century): Punched cards used to encode operations; later adopted by early computers.
- Machine code (1940s): Early electronic computers required binary instructions entered via switches or punched tape.

- Assembly language (1947): Kathleen Booth designs ARC assembly — symbolic mnemonics replacing raw binary.
- Still hardware-specific, but dramatically reduced cognitive load and enabled more complex programs.

- Autocode (1952): One of the first compiled languages.
- FORTRAN (1957): First widely used compiled high-level language; proved compilers could rival hand-written assembly.
- Lisp (1958): Introduced recursion, garbage collection, symbolic computation.
- COBOL (1959): English-like syntax for business computing; still used in global finance.

- ALGOL (1960): Introduced block structure and lexical scoping; ancestor of most modern languages.
- C (1972): Systems programming language enabling UNIX; portable, efficient, foundational.
- Smalltalk (1970s): Birth of object-oriented programming.

- Python (1991): Interpreted, dynamic, readable; later dominates data science.
- Java (1995): “Write once, run anywhere” via the JVM; major shift toward portable bytecode.
- JavaScript (1995): Becomes the de-facto language of the web.

- Rust (2010): Memory-safe systems programming; rising rapidly in 2020s.
- Go (2009): Concurrency-focused compiled language for cloud systems.
- TypeScript (2012): Typed superset of JavaScript for large-scale applications.

- AI-assisted coding (2021–): GitHub Copilot, Claude Code, Cursor reshape how code is written.
- AI-native languages (2023–2026): Emerging languages designed for LLM-driven development workflows.
- Programming shifts from writing code to composing intent, with AI generating scaffolding, tests, and transformations.

Why Programming Languages Evolved

Programming languages evolved through distinct eras, each introducing new abstractions that reshaped how humans communicate intent to machines.

Punched Cards &
Machine Code
(1840s–1950s)



Low-Level
Programming

Assembly Languages
(1947–1950s)



Symbolic Instructions

Early Compiled
High-Level Languages
(1952–1965)



High-Level Compilers

Structured
Programming
& Systems
Languages
(1969–1979)



C & Object-Oriented

Interpreted
Languages &
Virtual Machines
(1980s–2000s)



Web & Scripting

Modern Multi-Paradigm &
Memory-Safe Languages
(2010s–2020s)



Safe & Concurrent

AI-Assisted &
AI-Native
Programming
(2020s–2026)



AI-Driven
Development

Punched Cards → Assembly → Compiled Languages → Structured Languages → Interpreted Languages → Memory-Safe → AI-Assisted Programming

Need for faster,
less error-prone
coding.

Symbolic
mnemonics
replaced binary,
improving
readability.

Desire for
portability and
automation.

Compilers allowed
scientists and
engineers to focus
on logic, not
hardware.

Programs grew
complex; structure
reduced bugs.

ALGOL and C
introduced modular
design and control
flow clarity.

Need for rapid
development and
cross-platform
execution.

Interpreters
enabled
experimentation
and web scripting.

Security and
concurrency
became critical.

Rust and Go
addressed memory
leaks and race
conditions.

Explosion of
code complexity
and developer
tools.

AI now helps
write, refactor,
and test code —
shifting focus to
intent.

Why Python?

Python represents the moment when programming became high-level, expressive, portable, and accessible — the culmination of decades of abstraction.

Why Python?

The culmination of decades of abstraction.

1. Python = The Peak of Interpreted Languages

- Runs everywhere: laptops, servers, cloud, notebooks.
- No compilation step → immediate feedback.
- Ideal for teaching core concepts without ceremony.



PRINCETON
UNIVERSITY

Why Python?



The culmination of decades of abstraction.

2. Python = Human-Readable First

- Designed to read like English.
- Lets you focus on logic, not syntax.
- Bridges the gap between beginners and experts.



PRINCETON
UNIVERSITY

Why Python?



The culmination of decades of abstraction.

3. Python = Batteries Included

- Huge standard library.
- Rich ecosystems: NumPy, Pandas, Matplotlib, PyTorch.
- Perfect for scientific computing, data analysis, automation.

Why Python?

The culmination of decades of abstraction.

4. Python = The Language of AI

- Most machine learning frameworks are Python-first.
- AI tools (including Copilot) generate Python fluently.
- You learn a language that aligns with the future.

From Abstraction → Expression

- Everything in the timeline leads to Python's philosophy:
- “Let me express the idea, and let the language handle the details.”
- Python is the natural continuation of the abstraction arc — and the perfect foundation before you explore AI-assisted programming, intent-based coding, and software composition.

Python = The Language of Ideas

Python is where all the history becomes practical



Beautiful is better than ugly

Python values clean, elegant code.



Flat is better than nested

Deep nesting makes code harder to read.



Complex is better than complicated

Some complexity is necessary, but avoid tangled designs.



Sparse is better than dense.

Good spacing improves readability.



Explicit is better than implicit

Clarity beats cleverness; code should reveal intent.



Simple is better than complex

Prefer straightforward solutions.



Errors should never pass silently (unless explicitly silenced)

Python encourages safe, predictable behavior.



Readability counts

This is Python's most famous rule — code should be easy to understand.



There should be one — and preferably only one — obvious way to do it

Python encourages consistency and avoids unnecessary alternatives.



Namespaces are one honking great idea — let's do more of those!

Namespaces keep code organized and avoid conflicts.

Scientific Python Ecosystem

Batteries Included?

More Like a Fully Operational Power Grid.

